

Tugas 1 — RESTful API Murni dengan Express.js (SI5B)

A. Instruksi Pengerjaan

Setiap mahasiswa kelas SI5B membangun satu RESTful API murni dengan Express.js sesuai topik yang ditetapkan pada bagian C, tanpa tampilan (view/HTML) apa pun.

1. Tugas dikerjakan individu. Topik ditentukan berdasarkan nomor urut absen (absen 1 = Topik 1, dst. sampai absen 35). Setiap mahasiswa mendapat topik yang berbeda; tidak boleh bertukar topik, dan tidak boleh memakai resource contoh latihan (mahasiswa/students).
2. Gunakan Node.js (LTS) dan Express.js. Paket tambahan yang diperbolehkan hanya nodemon (dev). Tidak memakai template engine (EJS, Pug, dll.) dan tidak memakai database — data disimpan dalam array di memori (minimal 3 data awal).
3. Semua response wajib berformat JSON (`res.json()` / `res.status(...).json()`), termasuk response error dan 404. Dilarang memakai `res.send()` berisi teks/HTML atau `res.render()`.
4. Pengujian endpoint menggunakan Postman, Thunder Client, atau Insomnia — baik di localhost maupun di URL Vercel.
5. Project wajib di-push ke repository GitHub publik bernama `tugas1-restful-{nim}` (minimal 5 commit bertahap) dan di-deploy ke Vercel (lihat Soal 5).
6. Yang dikumpulkan ke LMS:
 - Laporan praktikum (PDF) dengan format pada bagian E, bernama `Laporan_Tugas1_NIM_Nama.pdf`.
 - Link repository GitHub dan link deploy Vercel (juga dicantumkan di laporan dan `README.md`).
 - File export koleksi Postman (`.json`) — opsional sebagai nilai tambah.
7. `README.md` di repository berisi: nama, NIM, nomor topik, link Vercel, cara menjalankan lokal, dan daftar endpoint.
8. Kode yang identik dengan mahasiswa lain (termasuk hanya mengganti nama resource) dinilai 0 untuk kedua pihak.
9. Batas pengumpulan mengikuti jadwal yang diumumkan di LMS. Keterlambatan dikenakan pengurangan nilai bertingkat sesuai tabel berikut.

Waktu pengumpulan	Pengurangan nilai	Nilai maksimal
Tepat waktu	0	100
Terlambat 1 hari (1–24 jam)	–10	90
Terlambat 2 hari (25–48 jam)	–20	80
Terlambat 3 hari (49–72 jam)	–30	70
Terlambat lebih dari 3 hari	Tidak diterima	0

Waktu pengumpulan dihitung dari waktu unggah terakhir di LMS; commit GitHub setelah batas waktu juga dianggap terlambat.

B. Deskripsi Tugas

RESTful API adalah layanan yang mengelola resource melalui URL dan method HTTP, lalu membalas dalam format data (di sini JSON), bukan halaman web. Pada tugas ini mahasiswa membuat operasi CRUD lengkap untuk satu resource sesuai topik, ditambah filter data dengan query string.

B.1 Konvensi Endpoint

Endpoint langsung memakai nama resource tanpa prefix, sama seperti latihan (`/mahasiswa`). Nama resource berupa kata benda jamak, huruf kecil, dipisah tanda hubung (contoh: `/coffee-orders`). Tidak boleh ada kata kerja di URL (salah: `/getToys`, `/deleteToy/1`).

No	Method	Endpoint	Fungsi	Status sukses	Status gagal
1	GET	<code>/ {resource}</code>	Ambil semua data	200	—
2	GET	<code>/ {resource} / :id</code>	Ambil satu data	200	404
3	POST	<code>/ {resource}</code>	Tambah data baru	201	400
4	PUT	<code>/ {resource} / :id</code>	Ubah seluruh data	200	400 / 404
5	DELETE	<code>/ {resource} / :id</code>	Hapus data	200	404
6	GET	<code>/ {resource} ? {filter} = nilai</code>	Filter dengan query string (req.query)	200	—

Ketentuan parameter: route parameter hanya dipakai untuk `:id` (`req.params.id`). Filter memakai query string dengan field pada kolom *Parameter filter* di bagian C (`req.query`), dan mengembalikan array langsung (boleh kosong `[]`) dengan status 200.

Ketentuan request:

- Body POST dan PUT dikirim sebagai JSON (`Content-Type: application/json`) dan dibaca dengan middleware `express.json()`.
- `id` dibuat otomatis oleh server (angka bertambah), tidak dikirim di body.
- Semua field bertanda *wajib* (*) pada rincian topik harus divalidasi; jika kosong, kirim status 400.
- PUT menerima seluruh field wajib (penggantian penuh).

B.2 Format Response JSON

Response GET mengembalikan data langsung tanpa status dan message, sama seperti latihan: objek untuk satu data (GET `/ {resource} / :id`), array untuk banyak data dan hasil filter.

```
{
  "id": 1,
  "nama": "Andi",
  "jurusan": "Sistem Informasi"
}
```

Response POST, PUT, DELETE, dan error memakai struktur berikut:

```
{
  "status": "success",
  "message": "Data berhasil ditambahkan",
  "data": { }
}
```

- `status`: "success" atau "error".
- `message`: kalimat singkat yang menjelaskan hasil.
- `data`: objek hasil POST/PUT, atau `null` (saat error / setelah hapus).

Contoh response error 404:

```
{
  "status": "error",
  "message": "Data dengan id 99 tidak ditemukan",
  "data": null
}
```

Contoh response error 400:

```
{
  "status": "error",
  "message": "Field nama wajib diisi",
  "data": null
}
```

Route yang tidak terdaftar ditangani middleware *catch-all* di akhir file dengan status 404 dan pesan "Endpoint tidak ditemukan".

B.3 Struktur Project

Seluruh kode ditulis dalam satu file `app.js` seperti latihan; belum perlu memecah ke folder `routes/controllers`.

```
tugas1-restful-{nim}/
├── .gitignore
├── vercel.json
├── package.json
├── app.js      # seluruh kode API
└── README.md
```

Urutan isi `app.js`: impor Express dan middleware `express.json()` → array data awal dan `nextId` → GET / → enam route (CRUD + filter) → *catch-all* 404 → `app.listen()` / `module.exports = app`.

B.4 Pengembangan dari Latihan (CRUD Mahasiswa)

Tugas 1 melanjutkan latihan CRUD /mahasiswa. Pola dasarnya boleh dipakai ulang: `express.json()`, array di memori, `nextId++`, `find/findIndex`, `req.query`, dan `req.params`. Bagian berikut wajib diubah agar sesuai rubrik:

Bagian	Pada latihan	Pada Tugas 1	Aspek
Route awal	GET / membalas teks dengan <code>res.send(...)</code>	GET / membalas JSON info API; tidak ada <code>res.send</code> teks	4
URL	/mahasiswa	{resource} sesuai topik (tetap tanpa prefix)	2
Response GET	Data langsung (<code>res.json(data)</code>)	Tetap data langsung (objek/array), tanpa status/message	4
Response POST/PUT/DELETE	Data langsung / kosong	Dibungkus { status, message, data }	4
Response error	{ message }	{ status: "error", message, data: null }	4
POST	Validasi 2 field	Validasi semua field wajib topik, status 201	3
PUT	Menggabung ... <code>req.body</code> tanpa validasi	Ganti penuh semua field wajib, 400 jika kosong, 404 jika id tidak ada	3
DELETE	204 tanpa body (<code>res.status(204).send()</code>)	200 + JSON berisi pesan dan data: null	3, 4
Endpoint tak dikenal	Balasan HTML bawaan Express	Middleware <i>catch-all</i> 404 berformat JSON	2, 4
Menjalankan server	<code>app.listen()</code> selalu jalan	<code>app.listen()</code> hanya lokal + <code>module.exports = app</code> (Vercel)	1

Contoh perubahan handler POST dari latihan ke format Tugas 1:

```
// POST /mahasiswa
// Body: { "nama": "Citra", "jurusan": "Sistem Informasi" }
app.post("/mahasiswa", (req, res) => {
  const { nama, jurusan } = req.body;

  // validasi: field wajib kosong -> 400
  if (!nama || !jurusan) {
    return res.status(400).json({
      status: "error",
```

```

        message: "nama dan jurusan wajib diisi",
        data: null,
    });
}

const baru = { id: nextId++, nama, jurusan };
mahasiswa.push(baru);

// berhasil -> 201 + data yang baru dibuat
res.status(201).json({
    status: "success",
    message: "Data berhasil ditambahkan",
    data: baru,
});
});
});

```

Response jika berhasil (201):

```

{
  "status": "success",
  "message": "Data berhasil ditambahkan",
  "data": {
    "id": 3,
    "nama": "Citra",
    "jurusan": "Sistem Informasi"
  }
}

```

Response jika field wajib kosong (400):

```

{
  "status": "error",
  "message": "nama dan jurusan wajib diisi",
  "data": null
}

```

Contoh perubahan handler DELETE dari latihan ke format Tugas 1:

```

// DELETE /mahasiswa/2
app.delete("/mahasiswa/:id", (req, res) => {
    const id = parseInt(req.params.id);
    const index = mahasiswa.findIndex(m => m.id === id);

    if (index === -1) {
        return res.status(404).json({ status: "error", message: `Data dengan id ${id} tidak ditemukan`, data: null });
    }

    mahasiswa.splice(index, 1);
    res.status(200).json({ status: "success", message: `Data mahasiswa dengan id ${id} berhasil dihapus`, data: null });
});

```

Komentar per baris seperti pada latihan (`// impor express`, `// Body: {...}`) tetap dianjurkan agar kode mudah dinilai.

C. Daftar Topik (35 Mahasiswa SI5B)

Setiap topik mencantumkan endpoint dasar, field dan tipe datanya (* = wajib), parameter filter, contoh data request (body POST/PUT), dan contoh output GET `/:id`. Semua endpoint 1–6 pada tabel B.1 wajib dibuat untuk topik masing-masing. Nilai output selain id hanyalah contoh.

Pembagian Topik per Nomor Absen

Absen	Nama mahasiswa	Topik	Resource	Endpoint dasar	Parameter filter
-------	----------------	-------	----------	----------------	------------------

Absen	Nama mahasiswa	Topik	Resource	Endpoint dasar	Parameter filter
1	Ahmad Andhika Pratama	Katering	Paket Katering	/catering-packages	jenisAcara
2	M. Alfatih Abdurrohlim	Kafe	Pesanan Kopi	/coffee-orders	ukuran
3	Sherafli Rusli	Toko Bunga	Rangkaian Bunga	/bouquets	jenisBunga
4	Nico Ramadhan Wijaya	Salon	Layanan Salon	/salon-services	kategori
5	M Darma Lopi	Kursus Bahasa	Kelas Bahasa	/language-classes	bahasa
6	Jassen Nova Tabalujan	Rental Alat Kemah	Peralatan Kemah	/camping-gears	kategori
7	Muhammad Luthfi Hilmy	PMI	Stok Darah	/blood-stocks	golonganDarah
8	Aisyah Ramadani	Posyandu	Data Balita	/toddlers	jenisKelamin
9	Athallah Zaki Cleon Alam	Perumahan	Iuran Warga	/resident-fees	bulan
10	Deisy Callisca Chandra	Kebun	Tanaman	/plants	jenis
11	Angga Syahputra Azis	Transportasi Umum	Rute Bus	/bus-routes	kota
12	Gerrard	Ekspedisi	Paket Kiriman	/shipments	statusPengiriman
13	Varrel Fernando	Museum	Koleksi Museum	/museum-collections	periode
14	M. Rizki Algipari	Galeri Seni	Lukisan	/paintings	aliran
15	Nurul Naima Sabandia	Toko Olahraga	Sepatu	/shoes	merk
16	Sunita Aqila	Kampus	Ruang Kelas	/classrooms	gedung
17	Muhammad Ammar Shaddiq	Kampus	Jadwal Ujian	/exam-schedules	ruang
18	Kirana Cinta Mentari	Kampus	Alumni	/alumni	tahunLulus
19	Jonathan Immanuel Alexander	Koperasi	Pinjaman Anggota	/loans	status
20	Muhammad Ilham Fajri	Asuransi	Polis	/insurance-policies	jenis
21	Putri Reda Maulidia	Kebencanaan	Laporan Bencana	/disaster-reports	jenisBencana
22	Arief Gunawan	Peternakan	Hewan Ternak	/livestock	jenis
23	Aldi Yonatan Rusnawan	Perikanan	Kolam Ikan	/fish-ponds	jenisIkan
24	Ewaldo Suhendra Kohar	Utilitas	Tagihan Listrik	/electricity-bills	golongan
25	Putri Angel Li	Telekomunikasi	Paket Data	/data-plans	operator
26	Fellisa Anastasya	Toko Gadget	Smartphone	/smartphones	merk
27	Olivia Wijaya	Toko Komputer	Laptop	/laptops	prosesor
28	Yonathan Rinfli	Coworking Space	Reservasi Ruang Rapat	/meeting-reservations	tanggal
29	Adelia	Studio Foto	Paket Foto	/photo-packages	jenis
30	Reva Lina	Sarana Olahraga	Booking Lapangan	/field-bookings	lapangan
31	Delia Zahrani	Puskesmas	Jadwal Vaksinasi	/vaccinations	jenisVaksin
32	Naufal Samih Najasyi	Kendaraan Listrik	Stasiun Pengisian	/charging-stations	kota
33	Kgs. Muhammad Alialjan Nama	Toko Tani	Pupuk	/fertilizers	jenis
34	Kurniawan	Kosmetik	Produk Skincare	/skincare-products	jenisKulit
35	Mas'ud Suhail	Toko Mainan	Mainan	/toys	kategoriUsia

Rincian field, data request, dan output setiap topik ada di bawah.

Topik 1 — Katering: Paket Katering (Ahmad Andhika Pratama)

- Endpoint dasar: /catering-packages
- Filter query string: GET /catering-packages?jenisAcara=pernikahan
- Field: namaPaket* string · jenisAcara* "pernikahan" | "ulang tahun" | "kantor" · hargaPerPorsi* number · minimalPorsi* number · menu array of string

Data request (POST/PUT):

```
{ "namaPaket": "Paket Melati", "jenisAcara": "pernikahan", "hargaPerPorsi": 45000, "minimalPorsi": 200, "menu": ["nasi putih", "ayam bakar", "sayur asem"] }
```

Output GET /catering-packages/1:

```
{
  "id": 1,
  "namaPaket": "Paket Melati",
  "jenisAcara": "pernikahan",
  "hargaPerPorsi": 45000,
  "minimalPorsi": 200,
  "menu": ["nasi putih", "ayam bakar", "sayur asem"]
}
```

Topik 2 — Kafe: Pesanan Kopi (M. Alfatih Abdurrohlim)

- Endpoint dasar: /coffee-orders
- Filter query string: GET /coffee-orders?ukuran=large
- Field: namaPelanggan* string · jenisKopi* string · ukuran* "small" | "medium" | "large" · jumlah* number · totalHarga* number

Data request (POST/PUT):

```
{ "namaPelanggan": "Rani", "jenisKopi": "Caffe Latte", "ukuran": "large", "jumlah": 2, "totalHarga": 64000 }
```

Output GET /coffee-orders/1:

```
{
  "id": 1,
  "namaPelanggan": "Rani",
  "jenisKopi": "Caffe Latte",
  "ukuran": "large",
  "jumlah": 2,
  "totalHarga": 64000
}
```

Topik 3 — Toko Bunga: Rangkaian Bunga (Sherafli Rusli)

- Endpoint dasar: /bouquets
- Filter query string: GET /bouquets?jenisBunga=mawar
- Field: nama* string · jenisBunga* string · warna string · harga* number · stok number

Data request (POST/PUT):

```
{ "nama": "Buket Wisuda Merah", "jenisBunga": "mawar", "warna": "merah", "harga": 175000, "stok": 8 }
```

Output GET /bouquets/1:

```
{
  "id": 1,
  "nama": "Buket Wisuda Merah",
  "jenisBunga": "mawar",
  "warna": "merah",
  "harga": 175000,
  "stok": 8
}
```

```
"stok": 8
}
```

Topik 4 — Salon: Layanan Salon (Nico Ramadhan Wijaya)

- Endpoint dasar: /salon-services
- Filter query string: GET /salon-services?kategori=rambut
- Field: namaLayanan* string · kategori* "rambut" | "wajah" | "kuku" · durasiMenit* number · harga* number · tersedia boolean

Data request (POST/PUT):

```
{ "namaLayanan": "Creambath", "kategori": "rambut", "durasiMenit": 60, "harga": 85000, "tersedia": true }
```

Output GET /salon-services/1:

```
{
  "id": 1,
  "namaLayanan": "Creambath",
  "kategori": "rambut",
  "durasiMenit": 60,
  "harga": 85000,
  "tersedia": true
}
```

Topik 5 — Kursus Bahasa: Kelas Bahasa (M Darma Lopi)

- Endpoint dasar: /language-classes
- Filter query string: GET /language-classes?bahasa=Jepang
- Field: namaKelas* string · bahasa* string · level* "dasar" | "menengah" | "lanjut" · jumlahPertemuan* number · biaya* number

Data request (POST/PUT):

```
{ "namaKelas": "Nihongo Pemula", "bahasa": "Jepang", "level": "dasar", "jumlahPertemuan": 16, "biaya": 1200000 }
```

Output GET /language-classes/1:

```
{
  "id": 1,
  "namaKelas": "Nihongo Pemula",
  "bahasa": "Jepang",
  "level": "dasar",
  "jumlahPertemuan": 16,
  "biaya": 1200000
}
```

Topik 6 — Rental Alat Kemah: Peralatan Kemah (Jassen Nova Tabalujan)

- Endpoint dasar: /camping-gears
- Filter query string: GET /camping-gears?kategori=tenda
- Field: namaAlat* string · kategori* "tenda" | "tas" | "masak" | "penerangan" · kapasitas number · hargaSewaPerHari* number · stok* number

Data request (POST/PUT):

```
{ "namaAlat": "Tenda Dome 4 Orang", "kategori": "tenda", "kapasitas": 4, "hargaSewaPerHari": 50000, "stok": 6 }
```

Output GET /camping-gears/1:

```
{
  "id": 1,
  "namaAlat": "Tenda Dome 4 Orang",
  "kategori": "tenda",
  "kapasitas": 4,
  "hargaSewaPerHari": 50000,
  "stok": 6
}
```

Topik 7 — PMI: Stok Darah (Muhammad Luthfi Hilmy)

- Endpoint dasar: /blood-stocks
- Filter query string: GET /blood-stocks?golonganDarah=0
- Field: golonganDarah* "A" | "B" | "AB" | "O" · rhesus* "+" | "-" · jumlahKantong* number · lokasi* string · tanggalUpdate* string (YYYY-MM-DD)

Data request (POST/PUT):

```
{ "golonganDarah": "O", "rhesus": "+", "jumlahKantong": 42, "lokasi": "UDD PMI Kota Palembang",
  "tanggalUpdate": "2026-09-28" }
```

Output GET /blood-stocks/1:

```
{
  "id": 1,
  "golonganDarah": "O",
  "rhesus": "+",
  "jumlahKantong": 42,
  "lokasi": "UDD PMI Kota Palembang",
  "tanggalUpdate": "2026-09-28"
}
```

Topik 8 — Posyandu: Data Balita (Aisyah Ramadani)

- Endpoint dasar: /toddlers
- Filter query string: GET /toddlers?jenisKelamin=P
- Field: nama* string · tanggalLahir* string (YYYY-MM-DD) · jenisKelamin* "L" | "P" · beratKg* number · tinggiCm number

Data request (POST/PUT):

```
{ "nama": "Alya", "tanggalLahir": "2024-05-12", "jenisKelamin": "P", "beratKg": 11.4,
  "tinggiCm": 82 }
```

Output GET /toddlers/1:

```
{
  "id": 1,
  "nama": "Alya",
  "tanggalLahir": "2024-05-12",
  "jenisKelamin": "P",
  "beratKg": 11.4,
  "tinggiCm": 82
}
```

Topik 9 — Perumahan: Iuran Warga (Athallah Zaki Cleon Alam)

- Endpoint dasar: /resident-fees
- Filter query string: GET /resident-fees?bulan=2026-09
- Field: namaWarga* string · nomorRumah* string · bulan* string (YYYY-MM) · nominal* number · lunas boolean

Data request (POST/PUT):

```
{ "namaWarga": "Pak Hasan", "nomorRumah": "B-12", "bulan": "2026-09", "nominal": 150000, "lunas": true }
```

Output GET /resident-fees/1:

```
{
  "id": 1,
  "namaWarga": "Pak Hasan",
  "nomorRumah": "B-12",
  "bulan": "2026-09",
  "nominal": 150000,
  "lunas": true
}
```

Topik 10 — Kebun: Tanaman (Deisy Callisca Chandra)

- Endpoint dasar: /plants
- Filter query string: GET /plants?jenis=sayur
- Field: namaTanaman* string · jenis* "sayur" | "buah" | "hias" · lokasiBedeng* string · tanggalTanam* string (YYYY-MM-DD) · sudahPanen boolean

Data request (POST/PUT):

```
{ "namaTanaman": "Kangkung", "jenis": "sayur", "lokasiBedeng": "Bedeng 3", "tanggalTanam": "2026-09-01", "sudahPanen": false }
```

Output GET /plants/1:

```
{
  "id": 1,
  "namaTanaman": "Kangkung",
  "jenis": "sayur",
  "lokasiBedeng": "Bedeng 3",
  "tanggalTanam": "2026-09-01",
  "sudahPanen": false
}
```

Topik 11 — Transportasi Umum: Rute Bus (Angga Syahputra Azis)

- Endpoint dasar: /bus-routes
- Filter query string: GET /bus-routes?kota=Palembang
- Field: kodeRute* string · asal* string · tujuan* string · kota* string · tarif* number

Data request (POST/PUT):

```
{ "kodeRute": "K1", "asal": "Terminal Alang-Alang Lebar", "tujuan": "Ampera", "kota": "Palembang", "tarif": 5000 }
```

Output GET /bus-routes/1:

```
{
  "id": 1,
  "kodeRute": "K1",
  "asal": "Terminal Alang-Alang Lebar",
  "tujuan": "Ampera",
  "kota": "Palembang",
  "tarif": 5000
}
```

Topik 12 — Ekspedisi: Paket Kiriman (Gerrard)

- Endpoint dasar: /shipments
- Filter query string: GET /shipments?statusPengiriman=dikirim
- Field: nomorResi* string · pengirim* string · penerima* string · beratKg* number · statusPengiriman* "diproses" | "dikirim" | "diterima"

Data request (POST/PUT):

```
{ "nomorResi": "EXP2609001", "pengirim": "Toko Sinar", "penerima": "Budi Hartono", "beratKg": 2.5, "statusPengiriman": "dikirim" }
```

Output GET /shipments/1:

```
{
  "id": 1,
  "nomorResi": "EXP2609001",
  "pengirim": "Toko Sinar",
  "penerima": "Budi Hartono",
  "beratKg": 2.5,
  "statusPengiriman": "dikirim"
}
```

Topik 13 — Museum: Koleksi Museum (Varrel Fernando)

- Endpoint dasar: /museum-collections
- Filter query string: GET /museum-collections?periode=Sriwijaya
- Field: namaKoleksi* string · kategori* string · periode* string · asalDaerah string · tahunDitemukan number

Data request (POST/PUT):

```
{ "namaKoleksi": "Arca Buddha Bukit Siguntang", "kategori": "arca", "periode": "Sriwijaya", "asalDaerah": "Palembang", "tahunDitemukan": 1920 }
```

Output GET /museum-collections/1:

```
{
  "id": 1,
  "namaKoleksi": "Arca Buddha Bukit Siguntang",
  "kategori": "arca",
  "periode": "Sriwijaya",
  "asalDaerah": "Palembang",
  "tahunDitemukan": 1920
}
```

Topik 14 — Galeri Seni: Lukisan (M. Rizki Algipari)

- Endpoint dasar: /paintings
- Filter query string: GET /paintings?aliran=realisme
- Field: judul* string · pelukis* string · aliran* string · tahunDibuat number · harga* number

Data request (POST/PUT):

```
{ "judul": "Senja di Musi", "pelukis": "Rahmat Hidayat", "aliran": "realisme", "tahunDibuat": 2023, "harga": 7500000 }
```

Output GET /paintings/1:

```
{
  "id": 1,
  "judul": "Senja di Musi",
  "pelukis": "Rahmat Hidayat",
  "aliran": "realisme",
  "tahunDibuat": 2023,
  "harga": 7500000
}
```

Topik 15 — Toko Olahraga: Sepatu (Nurul Naima Sabandia)

- Endpoint dasar: /shoes
- Filter query string: GET /shoes?merek=Ortuseight

- Field: namaProduk* string · merek* string · ukuran* number · harga* number · stok number

Data request (POST/PUT):

```
{ "namaProduk": "Sepatu Lari Hyperblast", "merek": "Ortuseight", "ukuran": 42, "harga": 529000, "stok": 10 }
```

Output GET /shoes/1:

```
{
  "id": 1,
  "namaProduk": "Sepatu Lari Hyperblast",
  "merek": "Ortuseight",
  "ukuran": 42,
  "harga": 529000,
  "stok": 10
}
```

Topik 16 — Kampus: Ruang Kelas (Sunita Aqila)

- Endpoint dasar: /classrooms
- Filter query string: GET /classrooms?gedung=A
- Field: kodeRuang* string · gedung* string · lantai number · kapasitas* number · adaProyektor boolean

Data request (POST/PUT):

```
{ "kodeRuang": "A-301", "gedung": "A", "lantai": 3, "kapasitas": 40, "adaProyektor": true }
```

Output GET /classrooms/1:

```
{
  "id": 1,
  "kodeRuang": "A-301",
  "gedung": "A",
  "lantai": 3,
  "kapasitas": 40,
  "adaProyektor": true
}
```

Topik 17 — Kampus: Jadwal Ujian (Muhammad Ammar Shaddiq)

- Endpoint dasar: /exam-schedules
- Filter query string: GET /exam-schedules?ruang=B-204
- Field: mataKuliah* string · tanggal* string (YYYY-MM-DD) · jamMulai* string (HH:mm) · ruang* string · pengawas string

Data request (POST/PUT):

```
{ "mataKuliah": "Pemrograman Web", "tanggal": "2026-11-02", "jamMulai": "08:00", "ruang": "B-204", "pengawas": "Dewi Anggraini, M.Kom" }
```

Output GET /exam-schedules/1:

```
{
  "id": 1,
  "mataKuliah": "Pemrograman Web",
  "tanggal": "2026-11-02",
  "jamMulai": "08:00",
  "ruang": "B-204",
  "pengawas": "Dewi Anggraini, M.Kom"
}
```

Topik 18 — Kampus: Alumni (Kirana Cinta Mentari)

- Endpoint dasar: /alumni
- Filter query string: GET /alumni?tahunLulus=2024
- Field: nim* string · nama* string · prodi* string · tahunLulus* number · pekerjaan string

Data request (POST/PUT):

```
{ "nim": "2020110045", "nama": "Farhan Akbar", "prodi": "Sistem Informasi", "tahunLulus": 2024, "pekerjaan": "Business Analyst" }
```

Output GET /alumni/1:

```
{
  "id": 1,
  "nim": "2020110045",
  "nama": "Farhan Akbar",
  "prodi": "Sistem Informasi",
  "tahunLulus": 2024,
  "pekerjaan": "Business Analyst"
}
```

Topik 19 — Koperasi: Pinjaman Anggota (Jonathan Immanuel Alexander)

- Endpoint dasar: /loans
- Filter query string: GET /loans?status=aktif
- Field: namaAnggota* string · jumlahPinjaman* number · tenorBulan* number · bungaPersen number · status* "aktif" | "lunas"

Data request (POST/PUT):

```
{ "namaAnggota": "Ibu Sari", "jumlahPinjaman": 5000000, "tenorBulan": 12, "bungaPersen": 1.5, "status": "aktif" }
```

Output GET /loans/1:

```
{
  "id": 1,
  "namaAnggota": "Ibu Sari",
  "jumlahPinjaman": 5000000,
  "tenorBulan": 12,
  "bungaPersen": 1.5,
  "status": "aktif"
}
```

Topik 20 — Asuransi: Polis (Muhammad Ilham Fajri)

- Endpoint dasar: /insurance-policies
- Filter query string: GET /insurance-policies?jenis=kesehatan
- Field: nomorPolis* string · namaPemegang* string · jenis* "kesehatan" | "jiwa" | "kendaraan" · premiBulanan* number · tanggalMulai* string (YYYY-MM-DD)

Data request (POST/PUT):

```
{ "nomorPolis": "POL-2026-0187", "namaPemegang": "Andi Wijaya", "jenis": "kesehatan", "premiBulanan": 350000, "tanggalMulai": "2026-01-01" }
```

Output GET /insurance-policies/1:

```
{
  "id": 1,
  "nomorPolis": "POL-2026-0187",
  "namaPemegang": "Andi Wijaya",
  "jenis": "kesehatan",
  "premiBulanan": 350000,
}
```

```
"tanggalMulai": "2026-01-01"
}
```

Topik 21 — Kebencanaan: Laporan Bencana (Putri Reda Maulidia)

- Endpoint dasar: /disaster-reports
- Filter query string: GET /disaster-reports?jenisBencana=banjir
- Field: jenisBencana* string | "longsor" | "kebakaran" | "gempa" · lokasi* string · tanggal* string (YYYY-MM-DD) · tingkatKeparahan* "ringan" | "sedang" | "berat" · jumlahPengungsi number

Data request (POST/PUT):

```
{ "jenisBencana": "banjir", "lokasi": "Kec. Seberang Ulu I", "tanggal": "2026-09-20",
  "tingkatKeparahan": "sedang", "jumlahPengungsi": 120 }
```

Output GET /disaster-reports/1:

```
{
  "id": 1,
  "jenisBencana": "banjir",
  "lokasi": "Kec. Seberang Ulu I",
  "tanggal": "2026-09-20",
  "tingkatKeparahan": "sedang",
  "jumlahPengungsi": 120
}
```

Topik 22 — Peternakan: Hewan Ternak (Arief Gunawan)

- Endpoint dasar: /livestock
- Filter query string: GET /livestock?jenis=sapi
- Field: kodeTernak* string · jenis* "sapi" | "kambing" | "ayam" · umurBulan* number · beratKg* number · sehat boolean

Data request (POST/PUT):

```
{ "kodeTernak": "SP-014", "jenis": "sapi", "umurBulan": 18, "beratKg": 320, "sehat": true }
```

Output GET /livestock/1:

```
{
  "id": 1,
  "kodeTernak": "SP-014",
  "jenis": "sapi",
  "umurBulan": 18,
  "beratKg": 320,
  "sehat": true
}
```

Topik 23 — Perikanan: Kolam Ikan (Aldi Yonatan Rusnawan)

- Endpoint dasar: /fish-ponds
- Filter query string: GET /fish-ponds?jenisIkan=lele
- Field: kodeKolam* string · jenisIkan* string · jumlahBibit* number · luasM2* number · tanggalTebar* string (YYYY-MM-DD)

Data request (POST/PUT):

```
{ "kodeKolam": "K-02", "jenisIkan": "lele", "jumlahBibit": 3000, "luasM2": 24, "tanggalTebar":
  "2026-08-15" }
```

Output GET /fish-ponds/1:

```
{
  "id": 1,
  "kodeKolam": "K-02",
  "jenisIkan": "lele",
  "jumlahBibit": 3000,
  "luasM2": 24,
  "tanggalTebar": "2026-08-15"
}
```

Topik 24 — Utilitas: Tagihan Listrik (Ewaldo Suhendra Kohar)

- Endpoint dasar: /electricity-bills
- Filter query string: GET /electricity-bills?golongan=R1
- Field: idPelanggan* string · nama* string · golongan* string · pemakaianKwh* number · totalTagihan* number

Data request (POST/PUT):

```
{ "idPelanggan": "171234567890", "nama": "Yudi Pratama", "golongan": "R1", "pemakaianKwh": 210,
  "totalTagihan": 303000 }
```

Output GET /electricity-bills/1:

```
{
  "id": 1,
  "idPelanggan": "171234567890",
  "nama": "Yudi Pratama",
  "golongan": "R1",
  "pemakaianKwh": 210,
  "totalTagihan": 303000
}
```

Topik 25 — Telekomunikasi: Paket Data (Putri Angel Li)

- Endpoint dasar: /data-plans
- Filter query string: GET /data-plans?operator=Nusanet
- Field: namaPaket* string · operator* string · kuotaGb* number · masaAktifHari* number · harga* number

Data request (POST/PUT):

```
{ "namaPaket": "Internet Bulanan 25GB", "operator": "Nusanet", "kuotaGb": 25, "masaAktifHari":
  30, "harga": 85000 }
```

Output GET /data-plans/1:

```
{
  "id": 1,
  "namaPaket": "Internet Bulanan 25GB",
  "operator": "Nusanet",
  "kuotaGb": 25,
  "masaAktifHari": 30,
  "harga": 85000
}
```

Topik 26 — Toko Gadget: Smartphone (Fellisa Anastasya)

- Endpoint dasar: /smartphones
- Filter query string: GET /smartphones?merek=Xiaomi
- Field: merek* string · model* string · ramGb* number · penyimpananGb* number · harga* number

Data request (POST/PUT):

```
{ "merek": "Xiaomi", "model": "Redmi Note 14", "ramGb": 8, "penyimpananGb": 256, "harga":
  3299000 }
```

Output GET /smartphones/1:

```
{
  "id": 1,
  "merek": "Xiaomi",
  "model": "Redmi Note 14",
  "ramGb": 8,
  "penyimpananGb": 256,
  "harga": 3299000
}
```

Topik 27 — Toko Komputer: Laptop (Olivia Wijaya)

- Endpoint dasar: /laptops
- Filter query string: GET /laptops?prosesor=Intel Core i5
- Field: merek* string · model* string · prosesor* string · ramGb* number · harga* number

Data request (POST/PUT):

```
{ "merek": "Lenovo", "model": "IdeaPad Slim 3", "prosesor": "Intel Core i5", "ramGb": 16,
  "harga": 8499000 }
```

Output GET /laptops/1:

```
{
  "id": 1,
  "merek": "Lenovo",
  "model": "IdeaPad Slim 3",
  "prosesor": "Intel Core i5",
  "ramGb": 16,
  "harga": 8499000
}
```

Topik 28 — Coworking Space: Reservasi Ruang Rapat (Yonathan Rinfi)

- Endpoint dasar: /meeting-reservations
- Filter query string: GET /meeting-reservations?tanggal=2026-10-05
- Field: namaPemesan* string · ruang* string · tanggal* string (YYYY-MM-DD) · jamMulai* string (HH:mm) · durasiJam* number

Data request (POST/PUT):

```
{ "namaPemesan": "PT Maju Digital", "ruang": "Ruang Kenanga", "tanggal": "2026-10-05",
  "jamMulai": "13:00", "durasiJam": 2 }
```

Output GET /meeting-reservations/1:

```
{
  "id": 1,
  "namaPemesan": "PT Maju Digital",
  "ruang": "Ruang Kenanga",
  "tanggal": "2026-10-05",
  "jamMulai": "13:00",
  "durasiJam": 2
}
```

Topik 29 — Studio Foto: Paket Foto (Adelia)

- Endpoint dasar: /photo-packages
- Filter query string: GET /photo-packages?jenis=wisuda
- Field: namaPaket* string · jenis* "wisuda" | "prewedding" | "produk" · jumlahFoto* number · durasiJam number · harga* number

Data request (POST/PUT):

```
{ "namaPaket": "Wisuda Keluarga", "jenis": "wisuda", "jumlahFoto": 20, "durasiJam": 1, "harga": 450000 }
```

Output GET /photo-packages/1:

```
{
  "id": 1,
  "namaPaket": "Wisuda Keluarga",
  "jenis": "wisuda",
  "jumlahFoto": 20,
  "durasiJam": 1,
  "harga": 450000
}
```

Topik 30 — Sarana Olahraga: Booking Lapangan (Reva Lina)

- Endpoint dasar: /field-bookings
- Filter query string: GET /field-bookings?lapangan=Futsal A
- Field: namaPemesan* string · lapangan* string · tanggal* string (YYYY-MM-DD) · jamMulai* string (HH:mm) · durasiJam* number

Data request (POST/PUT):

```
{ "namaPemesan": "Tim Garuda SI", "lapangan": "Futsal A", "tanggal": "2026-10-10", "jamMulai": "19:00", "durasiJam": 2 }
```

Output GET /field-bookings/1:

```
{
  "id": 1,
  "namaPemesan": "Tim Garuda SI",
  "lapangan": "Futsal A",
  "tanggal": "2026-10-10",
  "jamMulai": "19:00",
  "durasiJam": 2
}
```

Topik 31 — Puskesmas: Jadwal Vaksinasi (Delia Zahrani)

- Endpoint dasar: /vaccinations
- Filter query string: GET /vaccinations?jenisVaksin=Hepatitis B
- Field: namaPasien* string · nik* string · jenisVaksin* string · dosisKe* number · tanggal* string (YYYY-MM-DD)

Data request (POST/PUT):

```
{ "namaPasien": "Rudi Santoso", "nik": "1671010101010001", "jenisVaksin": "Hepatitis B", "dosisKe": 2, "tanggal": "2026-10-03" }
```

Output GET /vaccinations/1:

```
{
  "id": 1,
  "namaPasien": "Rudi Santoso",
  "nik": "1671010101010001",
  "jenisVaksin": "Hepatitis B",
  "dosisKe": 2,
  "tanggal": "2026-10-03"
}
```

Topik 32 — Kendaraan Listrik: Stasiun Pengisian (Naufal Samih Najasyi)

- Endpoint dasar: /charging-stations
- Filter query string: GET /charging-stations?kota=Palembang
- Field: nama* string · kota* string · jumlahSlot* number · dayaKw* number · buka24Jam boolean

Data request (POST/PUT):

```
{ "nama": "SPKLU Jakabaring", "kota": "Palembang", "jumlahSlot": 4, "dayaKw": 50, "buka24Jam": true }
```

Output GET /charging-stations/1:

```
{
  "id": 1,
  "nama": "SPKLU Jakabaring",
  "kota": "Palembang",
  "jumlahSlot": 4,
  "dayaKw": 50,
  "buka24Jam": true
}
```

Topik 33 — Toko Tani: Pupuk (Kgs. Muhammad Alialjan Nama)

- Endpoint dasar: /fertilizers
- Filter query string: GET /fertilizers?jenis=organik
- Field: namaPupuk* string · jenis* "organik" | "anorganik" · beratKg* number · harga* number · stok number

Data request (POST/PUT):

```
{ "namaPupuk": "Pupuk Kompos Super", "jenis": "organik", "beratKg": 25, "harga": 60000, "stok": 40 }
```

Output GET /fertilizers/1:

```
{
  "id": 1,
  "namaPupuk": "Pupuk Kompos Super",
  "jenis": "organik",
  "beratKg": 25,
  "harga": 60000,
  "stok": 40
}
```

Topik 34 — Kosmetik: Produk Skincare (Kurniawan)

- Endpoint dasar: /skincare-products
- Filter query string: GET /skincare-products?jenisKulit=berminyak
- Field: nama* string · merek* string · jenisKulit* "normal" | "kering" | "berminyak" | "sensitif" · harga* number · nomorBpom* string

Data request (POST/PUT):

```
{ "nama": "Gel Pembersih Wajah", "merek": "Kulitku", "jenisKulit": "berminyak", "harga": 49000, "nomorBpom": "NA18260100123" }
```

Output GET /skincare-products/1:

```
{
  "id": 1,
  "nama": "Gel Pembersih Wajah",
  "merek": "Kulitku",
  "jenisKulit": "berminyak",
  "harga": 49000,
  "nomorBpom": "NA18260100123"
}
```

Topik 35 — Toko Mainan: Mainan (Mas'ud Suhail)

- Endpoint dasar: /toys

- Filter query string: GET /toys?kategoriUsia=4-7
- Field: nama* string · kategoriUsia* "0-3" | "4-7" | "8+" · bahan string · harga* number · stok number

Data request (POST/PUT):

```
{ "nama": "Balok Susun Kayu", "kategoriUsia": "4-7", "bahan": "kayu", "harga": 95000, "stok": 15 }
```

Output GET /toys/1:

```
{
  "id": 1,
  "nama": "Balok Susun Kayu",
  "kategoriUsia": "4-7",
  "bahan": "kayu",
  "harga": 95000,
  "stok": 15
}
```

D. Soal Tugas 1

Kerjakan soal berikut untuk topik sesuai nomor absen Anda. Ganti {resource} dengan endpoint dasar topik Anda (contoh Topik 1: catering-packages).

Soal 1 — Membuat Project Express.js (Aspek 1)

1. Inisialisasi project dengan `npm init -y`, lalu pasang express sebagai *dependency* dan nodemon sebagai *devDependency*.
2. Atur entry point "main": "app.js", lalu tambahkan script "start": "node app.js" dan "dev": "nodemon app.js" pada package.json.
3. Buat satu file app.js yang menjalankan server pada port 3000 (atau dari process.env.PORT), memasang middleware express.json(), dan menampilkan log Server berjalan di `http://localhost:3000`.
4. Di app.js, buat array data sesuai topik dengan minimal 3 data awal dan variabel nextId untuk id berikutnya.
5. Buat route GET / yang mengembalikan JSON info API: nama mahasiswa, NIM, nomor topik, dan daftar endpoint.

Soal 2 — Routing RESTful (Aspek 2)

1. Tulis semua route langsung di app.js menggunakan app.get, app.post, app.put, dan app.delete, seperti latihan.
2. Terapkan 6 endpoint sesuai tabel B.1: route parameter hanya untuk :id, sedangkan filter memakai query string (req.query).
3. Beri komentar di atas setiap route berisi method, URL, dan contoh body (contoh: // POST /toys lalu // Body: { "nama": "..." }).
4. Tambahkan middleware *catch-all* 404 setelah semua route.
5. Pastikan penamaan URL mengikuti konvensi: kata benda jamak, huruf kecil, tanpa kata kerja.

Soal 3 — Penggunaan Method HTTP (Aspek 3)

1. GET hanya membaca data dan tidak mengubah array.
2. POST menambah data baru dari req.body; id dibuat otomatis (id terakhir + 1).
3. PUT mengganti seluruh field data dengan id tertentu; tolak jika data tidak ada (404) atau field wajib kosong (400).
4. DELETE menghapus data dengan id tertentu; tolak jika data tidak ada (404).

- Validasi semua field wajib (*) pada POST dan PUT sesuai rincian topik, termasuk nilai pilihan (contoh: "large", "organik") bila ada.
- Uji setiap method di Postman/Thunder Client dan simpan bukti ujinya, termasuk minimal 2 skenario gagal (404 dan 400).

Soal 4 — Response JSON Murni (Aspek 4)

- Response GET (semua data, satu data, dan filter) mengembalikan data langsung tanpa status/message: objek untuk satu data, array untuk banyak data.
- Response POST, PUT, DELETE, dan error memakai struktur { status, message, data } pada B.2.
- Gunakan status code yang tepat: 200 (GET/PUT/DELETE sukses), 201 (POST sukses), 400 (validasi gagal), 404 (data/endpoint tidak ditemukan).
- DELETE sukses mengembalikan data: null dan pesan yang menyebut id yang dihapus.
- Tidak ada `res.send()` teks/HTML, `res.render()`, atau file statis di seluruh project.

Contoh response GET /mahasiswa?jurusan=Informatika:

```
[
  {
    "id": 2,
    "nama": "Budi",
    "jurusan": "Informatika"
  }
]
```

Contoh response POST /mahasiswa (201):

```
{
  "status": "success",
  "message": "Data berhasil ditambahkan",
  "data": { "id": 3, "nama": "Citra", "jurusan": "Sistem Informasi" }
}
```

Contoh response DELETE /mahasiswa/3 (200):

```
{
  "status": "success",
  "message": "Data mahasiswa dengan id 3 berhasil dihapus",
  "data": null
}
```

Soal 5 — Push ke GitHub dan Deploy ke Vercel (Aspek 1)

- Buat file .gitignore berisi node_modules/ dan .env.
- Inisialisasi Git, lakukan commit bertahap minimal 5 kali dengan pesan yang jelas (contoh: init project express, tambah route GET, tambah POST dan validasi, tambah PUT dan DELETE, konfigurasi vercel).
- Push ke repository GitHub publik tugas1-restful-{nim}.
- Sesuaikan app.js agar bisa berjalan di Vercel (serverless): app di-export, dan app.listen() hanya dijalankan saat lokal.

```
const PORT = process.env.PORT || 3000;
if (process.env.NODE_ENV !== 'production') {
  app.listen(PORT, () => console.log(`Server berjalan di http://localhost:${PORT}`));
}
module.exports = app;
```

- Tambahkan file vercel.json di root project:

```
{
  "version": 2,
  "builds": [{ "src": "app.js", "use": "@vercel/node" }],
}
```

```
"routes": [{ "src": "/(.*)", "dest": "app.js" }]
```

6. Import repository dari GitHub di dashboard Vercel (*Add New* → *Project*), lalu deploy. Setiap push berikutnya ke branch `main` akan otomatis ter-deploy ulang.
7. Uji seluruh endpoint pada URL Vercel, contoh: `https://tugas1-restful-2301010001.vercel.app/catering-packages`.

Catatan: di Vercel, data array di memori dapat kembali ke data awal setelah beberapa saat (sifat *serverless*). Hal ini wajar dan tidak mengurangi nilai; cukup tunjukkan bahwa setiap request mendapat response yang benar.

E. Format Laporan Praktikum

Laporan disusun dalam PDF (A4, Times New Roman 12, spasi 1,5) mengikuti format laporan latihan sebelumnya, dengan isi minimal:

1. Halaman sampul — judul "Laporan Praktikum Tugas 1: RESTful API Murni dengan Express.js", nama, NIM, kelas (SI5B), nomor dan nama topik.
2. Tujuan praktikum — 3–4 poin yang merujuk pada keempat aspek penilaian.
3. Dasar teori singkat — REST dan resource, method HTTP (GET/POST/PUT/DELETE), HTTP status code, format JSON, dan Express.js (maks. 2 halaman, cantumkan sumber).
4. Alat dan bahan — versi Node.js, npm, Express, editor, tool uji API, akun GitHub dan Vercel.
5. Langkah praktikum — setiap langkah berisi penjelasan, potongan kode, dan screenshot: inisialisasi project, data awal, setiap route di `app.js`, validasi, middleware 404, push GitHub, deploy Vercel.
6. Hasil pengujian — tabel pengujian setiap endpoint di URL Vercel, disertai screenshot request + response + status code:

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	<code>/ {resource}</code>	—	200	200	Sesuai
2	GET	<code>/ {resource} /1</code>	—	200	...	...
3	GET	<code>/ {resource} /99</code>	—	404	...	...
4	GET	<code>/ {resource} ? {filter} =nilai</code>	—	200	...	...
5	POST	<code>/ {resource}</code>	body lengkap	201	...	...
6	POST	<code>/ {resource}</code>	field wajib kosong	400	...	...
7	PUT	<code>/ {resource} /1</code>	body lengkap	200	...	...
8	PUT	<code>/ {resource} /99</code>	body lengkap	404	...	...
9	DELETE	<code>/ {resource} /1</code>	—	200	...	...
10	DELETE	<code>/ {resource} /99</code>	—	404	...	...

7. Pembahasan — alasan pemilihan method dan status code, kendala yang ditemui (termasuk saat deploy), dan cara mengatasinya.
8. Kesimpulan — jawaban atas tujuan praktikum.
9. Lampiran — link repository GitHub, link Vercel, dan screenshot riwayat commit (`git log --oneline`).

F. Rubrik Penilaian

Nilai akhir = jumlah (skor aspek × bobot) ÷ 100. Laporan, link GitHub, dan link Vercel adalah syarat dinilai: tanpa salah satunya tugas dianggap belum dikumpulkan.

Aspek penilaian	Bobot	Sangat baik (86–100)	Baik (71–85)	Cukup (56–70)	Kurang (0–55)
-----------------	-------	----------------------	--------------	---------------	---------------

Aspek penilaian	Bobot	Sangat baik (86–100)	Baik (71–85)	Cukup (56–70)	Kurang (0–55)
1. Ketepatan membuat project berbasis Express.js	25%	package . json + script lengkap, app . js tersusun urut dan berkomentar, server jalan lokal dan di Vercel, repo GitHub ≥ 5 commit	Jalan lokal dan di Vercel, kode kurang rapi atau commit < 5	Jalan lokal, deploy Vercel gagal	Project tidak bisa dijalankan
2. Ketepatan pembuatan routing RESTful	25%	6 endpoint lengkap, : id dan filter query string benar, penamaan URL sesuai konvensi, ada handler 404	5–6 endpoint benar, 1–2 pelanggaran konvensi kecil	3–4 endpoint benar atau URL memakai kata kerja	≤ 2 endpoint atau routing tidak RESTful
3. Ketepatan penggunaan method HTTP	25%	GET/POST/PUT/DELETE sesuai fungsinya, validasi field wajib, id otomatis, kasus 404 dan 400 tertangani	Semua method benar, validasi belum lengkap	Ada method tertukar (contoh: hapus via GET) atau PUT/DELETE belum jalan	Hanya GET yang berfungsi
4. Ketepatan response murni JSON	25%	Semua response JSON: GET berupa data langsung, POST/PUT/DELETE/error memakai {status, message, data}, status code tepat (200/201/400/404), tidak ada res . send teks/HTML	Semua JSON, 1–2 format atau status code tidak sesuai	Sebagian response bukan JSON atau status code selalu 200	Response umumnya teks/HTML

Pengurangan nilai: laporan tidak mengikuti format bagian E (–10), screenshot pengujian tidak lengkap (–5 per endpoint), terlambat (bertingkat sesuai tabel keterlambatan di bagian A).